

Name:- Shubham Das

Date: - 29/09/2026

S. A. P I. D: - 590033500

Subject: - Programming In C

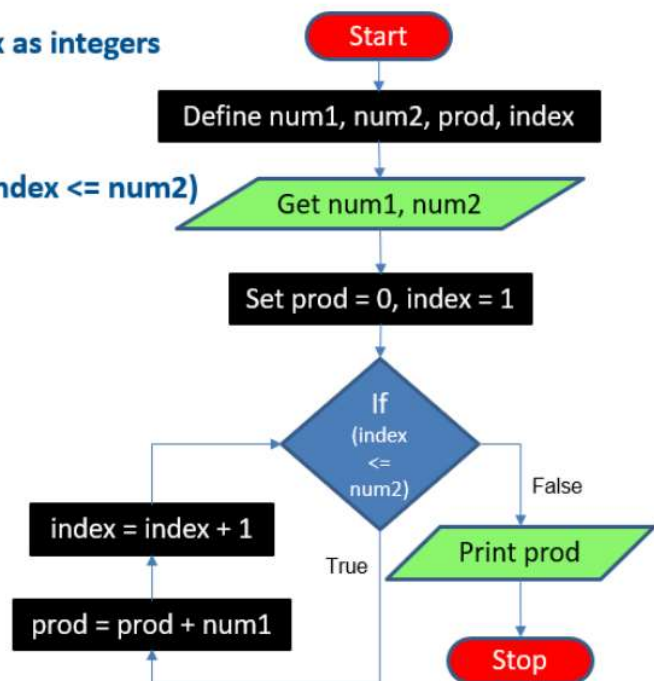
Objective: - Multiply 2 numbers without using *operator(9)

Algorithm & Flowchart: -

1. Define num1, num2, prod, index as integers
2. Get num1, num2
3. Set prod = 0, index = 1
4. Repeat steps 4.1 and 4.2 until (index <= num2)
 - 4.1 prod = prod + num1
 - 4.2. Index = index + 1
5. Print prod
6. Stop

• Algorithm & Flowchart

1. Define num1, num2, prod, index as integers
2. Get num1, num2
3. Set prod = 0, index = 1
4. Repeat steps 4.1 and 4.2 until (index <= num2)
 - 4.1. prod = prod + num1
 - 4.2. index = index + 1
5. Print prod
6. Stop



1. Mistakes in the Given Algorithm: -

1. The loop condition "index <= num2" fails entirely if "num2" is a negative integer (e.g., "1 <= -5" evaluates to false), causing the loop to skip and incorrectly output "0".
2. The algorithm does not calculate absolute values, making it incapable of handling negative multipliers or multiplicands properly as required by the test cases.

3. It fails to optimize for the minimum possible number of additions. For inputs like 3 and 12, it iterates 12 times adding 3, rather than iterating 12 times adding 12.

2. Corrected Algorithm: -

1. Define Variables: 'num1', 'num2', 'prod', 'index', 'abs1', 'abs2', 'min_val', 'max_val' as integers.
2. Get 'num1', 'num2'.
3. Set 'prod = 0', 'index = 1'.
4. Calculate absolute values: 'abs1 = |num1|', 'abs2 = |num2|'.
5. Check if 'abs1' < 'abs2':
 - 5.1 If False: Set min_val = abs1
max_val = abs2
 - 5.2 If True: Set min_val = abs2
max_val = abs1
6. Repeat steps 6.1 and 6.2 until (index <= min_val):
 - 6.1 prod = prod + max_val
 - 6.2 index = index + 1
7. Check if ('num1 < 0' AND 'num2 < 0')
OR ('num1 > 0' AND 'num2 < 0'):
 - 7.1 If True Set: Set prod = -prod
8. Print prod
9. Stop.

3. Mistakes in the Given Flowchart

1. If (index <= num2) will branch to "False" immediately for negative numbers.
2. There are no process blocks to determine absolute values or to dynamically swap values to ensure the smaller number acts as the loop counter.
3. There is no decision logic at the end to correct the final mathematical sign based on whether the input combination was positive/negative or negative/positive.

4. Corrected Flowchart Instructions

Start =>

Input: Get num1, num2 =>

Process: Set prod = 0, index = 1, abs1 = |num1| =>

Decision: Is abs1 < abs2? =>

True Arrow-> Process (min_val = abs1, max_val = abs2)

False Arrow-> Process (min_val = abs2, max_val = abs1)

Both branches converge to a new decision: Is index <= min_val?

True Arrow -> Process: "prod = prod + max_val" followed by "index = index + 1". (Loop this back to the "index <= min_val")

False Arrow -> Decision: Is (num1 < 0 AND num2 > 0) OR (num1 > 0)

True Arrow -> Process: prod = -prod

-> Connect to Output

False Arrow -> Connect directly to Output

Output = "Print prod"

Stop

5. Verification with Given Values

The corrected logic successfully achieves the minimum additions by isolating the smaller absolute value (min_val) for the loop threshold.

Test	Inputs (num1, num2)	Min_val (Loop Count)	Max_val (Added Value)	Sign Correction Needed?	Final Product	Minimum Additions
1	12, 3	3	12	No	36	3
2	3, 12	3	12	No	36	3
3	-6, 5	5	6	Yes	-30	5
5	-9, -4	4	9	No	36	4
7	0, 8	0	8	No	0	0
12	1, -7	1	7	Yes	-7	1